A Lab Manual For

DATA SCIENCE LAB

25MMCA019HY

A RECORD SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR
"DATA SCIENCE LAB" OF

MASTER OF COMPUTER APPLICATION

2ND Year / 3RD Semester

2025-26

By

SALEH HAYAT

Roll Number: 25MMCA019HY

Enrolment Number: A250017

Submitted to

Mr. Kamran Siddiqui

Assistant Professor

Department of CS & IT, MANUU



SCHOOL OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

MAULANA AZAD NATIONAL URDU UNIVERSITY

(A Central University established by an Act of Parliament in 1998)

Accredited Grade "A+" by NAAC

Gachibowli, Hyderabad-500032, T.S. www.manuu.edu.in

DEPARTMENT OF
DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

Certificate

Certified that this is the Bonafide **DATA SCIENCE Lab (MMCA311PCT)** Record of Mr./Ms. **Saleh Hayat**, Roll No **25MMCA019HY**, MCA II Year, III Semester, of the academic year 2025-26.

Head,
Department of CS & IT

Internal Examiner

External Examiner

INDEX / انڈیکس

LAB PRACTICE RECORD / ریکارڈ عملی لیب

S.No. سیریل نمبر	Date تاریخ	Program Name پروگرام کا نام	Page No. صفحہ نمبر	Remarks ریمارکس
1	2025-08-18	**Question 1: Pandas Dataset Creation & Export** Create a dataset of students with Name, Department, Marks, and Age, convert into DataFrame, and save to students.csv.	5	Completed
2	2025-08-25	**Question 2: Pandas Data Operations & Filtering** Perform operations on students.csv: display head/tail, column selection, marks filter (>75), Grade column, missing values, sorting, groupby.	6	Completed
3	2025-09-01	**Question 3: Dataset Aggregations & Distribution Analysis** Analyze students.csv: total student count, highest & lowest marks, department breakdown, grade distributions, marks filtering (60-80).	10	Completed
4	2025-09-08	**Question 4: Multi-Table Merging & Relational Operations** Merge students.csv and fees.csv DataFrames on common Name, filter pending fees, paid fees with marks > 75, average marks by fee status.	14	Completed
5	2025-09-15	**Question 5: Descriptive Statistics & Data Visualization** Comprehensive statistical analysis for 30 students dataset: Central tendency, Dispersion, Quartiles, Skewness/Kurtosis, Covariance, Correlation, Outlier detection & Visualizations.	18	Completed

INDEX / انڈیکس (Continued)

LAB PRACTICE RECORD / ریکارڈ عملی لیب

S.No. سیریل نمبر	Date تاریخ	Program Name پروگرام کا نام	Page No. صفحہ نمبر	Remarks ریمارکس
6	2025-09-22	**Question 6: Simple Linear Regression** Implementation of Simple Linear Regression using scikit-learn, train/test split, MSE and R2 evaluation, prediction for 7.5 study hours, and regression line visualization.	27	Completed
7	2025-09-29	**Question 7: Multiple Linear Regression** Predict student exam scores using study hours, attendance and assignment scores, evaluate model performance, and inspect model coefficients and intercept.	30	Completed
8	2025-10-06	**Question 8: Logistic Regression Classification** Binary classification model using Logistic Regression to predict Pass or Fail status based on study hours and attendance, with accuracy and confusion matrix evaluation.	33	Completed
9	2025-10-13	**Question 9: Decision Tree Classification** Decision Tree Classifier to predict student outcome based on study hours, attendance, and assignment score, with visual tree rendering and test predictions.	36	Completed

Question 1: Pandas: Create Student Dataset & CSV Export

Problem Statement:

Create a dataset of students with Name, Department, Marks, and Age, convert into DataFrame, and save to students.csv.

PYTHON CODE:

```
import pandas as pd

data = {
    "Name": ["Amit","Riya","Rahul","Sneha","Arjun","Pooja","Karan","Neha","Vikas","Anjali"],
    "Department": ["CSE","IT","ECE","CSE","IT","ECE","CSE","IT","ECE","CSE"],
    "Marks": [85,78,65,90,55,72,88,60,45,82],
    "Age": [20,21,22,20,23,21,22,20,23,21]
}

df = pd.DataFrame(data)
df.to_csv("students.csv", index=False)

print("students.csv file created successfully!")
```

```
students.csv file created successfully!
```

PYTHON CODE:

```
pd.read_csv("students.csv")
```

```
  Name Department  Marks  Age
0    Amit        CSE     85   20
1    Riya         IT     78   21
2   Rahul        ECE     65   22
3   Sneha        CSE     90   20
4   Arjun         IT     55   23
5   Pooja        ECE     72   21
6   Karan        CSE     88   22
7    Neha         IT     60   20
8   Vikas        ECE     45   23
9  Anjali        CSE     82   21
```

Question 2: Pandas: Operations on CSV File (Filtering, Sorting, Grouping)

> **Problem Statement:**
> Perform operations on students.csv: display head/tail, column selection, marks filter (>75), Grade column, missing values, sorting, groupby.

PYTHON CODE:

```
# 1. Read CSV file and display first 5 rows
df = pd.read_csv("students.csv")
print("First 5 rows:\n", df.head())
```

```
First 5 rows:
     Name Department  Marks  Age
0   Amit        CSE     85   20
1   Riya         IT     78   21
2  Rahul        ECE     65   22
3  Sneha        CSE     90   20
4  Arjun         IT     55   23
```

PYTHON CODE:

```
# 2. Display last 3 rows and dataset info
print("\nLast 3 rows:\n", df.tail(3))
print("\nDataset Info:")
print(df.info())
```

```
Last 3 rows:
      Name Department  Marks  Age
7    Neha         IT     60   20
8   Vikas        ECE     45   23
9  Anjali        CSE     82   21

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Name        10 non-null     object
 1   Department  10 non-null     object
 2   Marks       10 non-null     int64
 3   Age         10 non-null     int64
dtypes: int64(2), object(2)
memory usage: 452.0+ bytes
None
```

PYTHON CODE:

```
# 3. Select specific columns
print("\nSelected Columns (Name, Marks):\n", df[['Name', 'Marks']])
```

```
Selected Columns (Name, Marks):
      Name  Marks
0    Amit     85
1    Riya     78
2   Rahul     65
3   Sneha     90
4   Arjun     55
5   Pooja     72
6   Karan     88
7    Neha     60
8   Vikas     45
9  Anjali     82
```

PYTHON CODE:

```
# 4. Filter students with marks > 75
high_marks = df[df['Marks'] > 75]
print("\nStudents with Marks > 75:\n", high_marks)
```

```
Students with Marks > 75:
      Name Department  Marks  Age
0    Amit        CSE     85   20
1    Riya         IT     78   21
3   Sneha        CSE     90   20
6   Karan        CSE     88   22
9  Anjali        CSE     82   21
```

PYTHON CODE:

```
# 5. Add Grade column
def grade(marks):
    if marks ⩾ 80:
        return 'A'
    elif marks ⩾ 60:
        return 'B'
    else:
        return 'C'

df['Grade'] = df['Marks'].apply(grade)
print("\nData with Grade:\n", df)
```

```
Data with Grade:
      Name Department  Marks  Age Grade
0    Amit        CSE     85   20     A
1    Riya         IT     78   21     B
2   Rahul        ECE     65   22     B
3   Sneha        CSE     90   20     A
4   Arjun         IT     55   23     C
5   Pooja        ECE     72   21     B
6   Karan        CSE     88   22     A
7    Neha         IT     60   20     B
8   Vikas        ECE     45   23     C
9  Anjali        CSE     82   21     A
```

PYTHON CODE:

```
# 6. Handle missing values
print("\nMissing Values:\n", df.isnull().sum())
df.fillna(0, inplace=True)
```

```
Missing Values:
 Name          0
Department    0
Marks         0
Age           0
Grade         0
dtype: int64
```

PYTHON CODE:

```
# 7. Sort data by Marks (descending)
sorted_df = df.sort_values(by='Marks', ascending=False)
print("\nSorted Data:\n", sorted_df)
```

```
Sorted Data:
      Name Department  Marks  Age Grade
3   Sneha        CSE     90   20     A
6   Karan        CSE     88   22     A
0    Amit        CSE     85   20     A
9  Anjali        CSE     82   21     A
1    Riya         IT     78   21     B
5   Pooja        ECE     72   21     B
2   Rahul        ECE     65   22     B
7    Neha         IT     60   20     B
4   Arjun         IT     55   23     C
8   Vikas        ECE     45   23     C
```

PYTHON CODE:

```
# 8. Group by Department and find average marks
grouped = df.groupby('Department')['Marks'].mean()
print("\nAverage Marks by Department:\n", grouped)
```

```
Average Marks by Department:
 Department
CSE    86.250000
ECE    60.666667
IT     64.333333
Name: Marks, dtype: float64
```

PYTHON CODE:

```
# 9. Save updated data to new CSV
df.to_csv("Students.csv", index=False)
```

PYTHON CODE:

```
# 10. Statistical summary
print("\nStatistical Summary:\n", df.describe())
```

```
Statistical Summary:
            Marks        Age
count  10.000000  10.000000
mean   72.000000  21.300000
std    15.260698   1.159502
min    45.000000  20.000000
25%    61.250000  20.250000
50%    75.000000  21.000000
75%    84.250000  22.000000
max    90.000000  23.000000
```

Question 3: Pandas: Student Dataset Statistical & Department Analysis

Problem Statement:

Analyze students.csv: total student count, highest & lowest marks, department breakdown, grade distributions, marks filtering (60-80).

PYTHON CODE:

```
# 1. Read the CSV file
df = pd.read_csv("Students.csv")

print("Student Dataset:\n")
print(df)
```

```
Student Dataset:

     Name Department  Marks  Age Grade
0    Amit        CSE     85   20     A
1    Riya         IT     78   21     B
2   Rahul        ECE     65   22     B
3   Sneha        CSE     90   20     A
4   Arjun         IT     55   23     C
5   Pooja        ECE     72   21     B
6   Karan        CSE     88   22     A
7    Neha         IT     60   20     B
8   Vikas        ECE     45   23     C
9  Anjali        CSE     82   21     A
```

PYTHON CODE:

```
# 2. Display total number of students
print("\nTotal Number of Students:", len(df))
```

```
Total Number of Students: 10
```

PYTHON CODE:

```
# 3. Find student with highest marks
highest = df.loc[df['Marks'].idxmax()]
print("\nStudent with Highest Marks:")
print(highest)
```

```
Student with Highest Marks:
Name          Sneha
Department      CSE
Marks            90
Age              20
Grade             A
Name: 3, dtype: object
```

PYTHON CODE:

```
# 4. Find student with lowest marks
lowest = df.loc[df['Marks'].idxmin()]
print("\nStudent with Lowest Marks:")
print(lowest)
```

```
Student with Lowest Marks:
Name          Vikas
Department      ECE
Marks            45
Age              23
Grade             C
Name: 8, dtype: object
```

PYTHON CODE:

```
# 5. Calculate average, maximum and minimum marks
print("\nAverage Marks:", df['Marks'].mean())
print("Maximum Marks:", df['Marks'].max())
print("Minimum Marks:", df['Marks'].min())
```

```
Average Marks: 72.0
Maximum Marks: 90
Minimum Marks: 45
```

PYTHON CODE:

```
# 6. Count students in each Department
print("\nNumber of Students in Each Department:")
print(df['Department'].value_counts())
```

```
Number of Students in Each Department:
Department
CSE    4
IT     3
ECE    3
Name: count, dtype: int64
```

PYTHON CODE:

```
# 7. Count students in each Grade
print("\nNumber of Students in Each Grade:")
print(df['Grade'].value_counts())
```

```
Number of Students in Each Grade:
Grade
A    4
B    4
C    2
Name: count, dtype: int64
```

PYTHON CODE:

```
# 8. Display students with marks between 60 and 80
students_60_80 = df[df['Marks'].between(60, 80)]

print("\nStudents with Marks Between 60 and 80:")
print(students_60_80)
```

```
Students with Marks Between 60 and 80:
    Name Department  Marks  Age Grade
1   Riya         IT     78   21     B
2  Rahul        ECE     65   22     B
5  Pooja        ECE     72   21     B
7   Neha         IT     60   20     B
```

PYTHON CODE:

```
# 9. Display students belonging to CSE department
cse_students = df[df['Department'] == 'CSE']

print("\nCSE Students:")
print(cse_students[['Name', 'Marks', 'Age', 'Grade']])
```

```
CSE Students:
     Name  Marks  Age Grade
0    Amit     85   20     A
3   Sneha     90   20     A
6   Karan     88   22     A
9  Anjali     82   21     A
```

PYTHON CODE:

```
# 10. Save students scoring 75 or more
top_students = df[df['Marks'] >= 75]

top_students.to_csv("students_75_plus.csv", index=False)

print("\nstudents_75_plus.csv file created successfully!")
```

```
students_75_plus.csv file created successfully!
```

Question 4: Pandas: Multi-file Merge (students.csv & fees.csv)

Problem Statement:

Merge students.csv and fees.csv DataFrames on common Name, filter pending fees, paid fees with marks > 75, average marks by fee status.

PYTHON CODE:

```
import pandas as pd

fees_data = {
    "Name": ["Amit", "Riya", "Rahul", "Sneha", "Arjun",
             "Pooja", "Karan", "Neha", "Vikas", "Anjali"],
    "Fees_Status": ["Paid", "Pending", "Paid", "Paid", "Pending",
                    "Paid", "Paid", "Pending", "Pending", "Paid"]
}

fees_df = pd.DataFrame(fees_data)

fees_df.to_csv("fees.csv", index=False)

print("fees.csv file created successfully!")
```

```
fees.csv file created successfully!
```

PYTHON CODE:

```
# 1) Read both CSV files
students = pd.read_csv("students.csv")
fees = pd.read_csv("fees.csv")
```

PYTHON CODE:

```
# 2) Display both DataFrames
print("Students Data:")
print(students)

print("\nFees Data:")
print(fees)
```

```
Students Data:
     Name Department  Marks  Age Grade
0    Amit        CSE     85   20     A
1    Riya         IT     78   21     B
2   Rahul        ECE     65   22     B
3   Sneha        CSE     90   20     A
4   Arjun         IT     55   23     C
5   Pooja        ECE     72   21     B
6   Karan        CSE     88   22     A
7    Neha         IT     60   20     B
8   Vikas        ECE     45   23     C
9  Anjali        CSE     82   21     A

Fees Data:
     Name Fees_Status
0    Amit        Paid
1    Riya     Pending
2   Rahul        Paid
3   Sneha        Paid
4   Arjun     Pending
5   Pooja        Paid
6   Karan        Paid
7    Neha     Pending
8   Vikas     Pending
9  Anjali        Paid
```

PYTHON CODE:

```
# 3) Merge both DataFrames using Name
merged_df = pd.merge(students, fees, on="Name")

print("\nMerged Data:")
print(merged_df)
```

```
Merged Data:
     Name Department  Marks  Age Grade Fees_Status
0    Amit        CSE     85   20     A        Paid
1    Riya         IT     78   21     B     Pending
2   Rahul        ECE     65   22     B        Paid
3   Sneha        CSE     90   20     A        Paid
4   Arjun         IT     55   23     C     Pending
5   Pooja        ECE     72   21     B        Paid
6   Karan        CSE     88   22     A        Paid
7    Neha         IT     60   20     B     Pending
8   Vikas        ECE     45   23     C     Pending
9  Anjali        CSE     82   21     A        Paid
```

PYTHON CODE:

```
# 4) Display selected columns
print("\nSelected Columns:")
print(merged_df[["Name", "Department", "Marks", "Fees_Status"]])
```

```
Selected Columns:
     Name Department  Marks Fees_Status
0    Amit        CSE     85        Paid
1    Riya         IT     78     Pending
2   Rahul        ECE     65        Paid
3   Sneha        CSE     90        Paid
4   Arjun         IT     55     Pending
5   Pooja        ECE     72        Paid
6   Karan        CSE     88        Paid
7    Neha         IT     60     Pending
8   Vikas        ECE     45     Pending
9  Anjali        CSE     82        Paid
```

PYTHON CODE:

```
# 5) Find students whose fees are Pending
pending = merged_df[merged_df["Fees_Status"] == "Pending"]

print("\nStudents with Pending Fees:")
print(pending[["Name", "Department", "Marks", "Fees_Status"]])
```

```
Students with Pending Fees:
    Name Department  Marks Fees_Status
1   Riya         IT     78     Pending
4  Arjun         IT     55     Pending
7   Neha         IT     60     Pending
8  Vikas        ECE     45     Pending
```

PYTHON CODE:

```
# 6) Students with marks > 75 and Paid fees
top_paid = merged_df[
    (merged_df["Marks"] > 75) &
    (merged_df["Fees_Status"] == "Paid")
]

print("\nStudents with Marks > 75 and Paid Fees:")
print(top_paid[["Name", "Marks", "Fees_Status"]])
```

```
Students with Marks > 75 and Paid Fees:
     Name  Marks Fees_Status
0    Amit     85        Paid
3   Sneha     90        Paid
6   Karan     88        Paid
9  Anjali     82        Paid
```

PYTHON CODE:

```
# 7) Count students according to Fees_Status
print("\nFees Status Count:")
print(merged_df["Fees_Status"].value_counts())
```

```
Fees Status Count:
Fees_Status
Paid       6
Pending    4
Name: count, dtype: int64
```

PYTHON CODE:

```
# 8) Average marks according to Fees_Status
print("\nAverage Marks by Fees Status:")
print(merged_df.groupby("Fees_Status")["Marks"].mean())
```

```
Average Marks by Fees Status:
Fees_Status
Paid       80.333333
Pending    59.500000
Name: Marks, dtype: float64
```

PYTHON CODE:

```
# 9) Sort by Marks in descending order
sorted_df = merged_df.sort_values(
    by="Marks",
    ascending=False
)

print("\nData Sorted by Marks:")
print(sorted_df)
```

```
Data Sorted by Marks:
     Name Department  Marks  Age Grade Fees_Status
3   Sneha        CSE     90   20     A        Paid
6   Karan        CSE     88   22     A        Paid
0    Amit        CSE     85   20     A        Paid
9  Anjali        CSE     82   21     A        Paid
1    Riya         IT     78   21     B     Pending
5   Pooja        ECE     72   21     B        Paid
2   Rahul        ECE     65   22     B        Paid
7    Neha         IT     60   20     B     Pending
4   Arjun         IT     55   23     C     Pending
8   Vikas        ECE     45   23     C     Pending
```

PYTHON CODE:

```
# 10) Save merged DataFrame
merged_df.to_csv(
    "student_fee_details.csv",
    index=False
)

print("\nstudent_fee_details.csv file created successfully!")
```

```
student_fee_details.csv file created successfully!
```

Question 5: Statistics for Data Science (Mean, Median, IQR, Skewness, Plots)

Problem Statement:

Comprehensive statistical analysis for 30 students dataset: Central tendency, Dispersion, Quartiles, Skewness/Kurtosis, Covariance, Correlation, Outlier detection & Visualizations.

PYTHON CODE:

```
import pandas as pd
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt

# Create dataset of 30 students

np.random.seed(42)
df = pd.DataFrame({
    'Student_ID': range(1, 31),
    'Age': np.random.randint(18, 24, 30),
    'Study_Hours': np.random.randint(2, 10, 30),
    'Attendance': np.random.randint(60, 101, 30),
    'Assignment_Score': np.random.randint(50, 101, 30),
    'Exam_Score': np.random.randint(45, 101, 30) })
print(df)
```

	Student_ID	Age	Study_Hours	Attendance	Assignment_Score	Exam_Score	
0	1	21	4	68	65	83	
1	2	22	8	85	94	85	
2	3	20	3	61	67	72	
3	4	22	9	79	96	51	
4	5	22	5	87	73	53	
5	6	19	5	66	75	52	
6	7	20	9	67	74	56	
7	8	20	8	94	94	78	
8	9	20	7	73	90	77	
9	10	22	7	76	78	92	
10	11	21	8	95	64	99	
11	12	20	7	99	94	67	
12	13	23	4	63	50	68	
13	14	22	5	61	74	81	
14	15	19	8	65	56	79	
15	16	21	5	63	58	88	
16	17	23	9	88	73	84	
17	18	23	2	77	50	66	
18	19	19	4	85	93	71	
19	20	21	6	93	57	79	
20	21	22	4	69	73	45	
21	22	18	8	95	60	79	
22	23	21	6	73	100	81	
23	24	19	2	90	66	91	
24	25	23	8	74	57	58	
25	26	22	3	67	84	47	
26	27	21	5	73	84	45	
27	28	18	2	82	82	49	
28	29	18	5	99	54	70	
29	30	20	7	80	91	99	

PYTHON CODE:

```
# 1. Mean, Median and Mode
print("Mean:\n", df.mean(numeric_only=True))
print("Median:\n", df.median(numeric_only=True))
print("Mode:\n", df.mode(numeric_only=True).iloc[0])
```

```
Mean:
 Student_ID          15.500000
Age                 20.733333
Study_Hours          5.766667
Attendance          78.233333
Assignment_Score    74.200000
Exam_Score          71.500000
dtype: float64
Median:
 Student_ID          15.5
Age                 21.0
Study_Hours          5.5
Attendance          76.5
Assignment_Score    73.5
Exam_Score          74.5
dtype: float64
Mode:
 Student_ID           1.0
Age                 22.0
Study_Hours          5.0
Attendance          73.0
Assignment_Score    73.0
Exam_Score          79.0
Name: 0, dtype: float64
```

PYTHON CODE:

```
# 2. Minimum, Maximum, Range and Sum
print("Minimum:\n", df.min(numeric_only=True))
print("Maximum:\n", df.max(numeric_only=True))
print("Range:\n", df.max(numeric_only=True) - df.min(numeric_only=True))
print("Sum:\n", df.sum(numeric_only=True))
```

```
Minimum:
 Student_ID          1
Age                 18
Study_Hours          2
Attendance          61
Assignment_Score    50
Exam_Score          45
dtype: int64
Maximum:
 Student_ID         30
Age                 23
Study_Hours          9
Attendance          99
Assignment_Score   100
Exam_Score          99
dtype: int64
Range:
 Student_ID         29
Age                  5
Study_Hours          7
Attendance          38
Assignment_Score    50
Exam_Score          54
dtype: int64
Sum:
 Student_ID         465
Age                 622
Study_Hours         173
Attendance         2347
Assignment_Score   2226
Exam_Score         2145
dtype: int64
```

PYTHON CODE:

```
# 3. Variance and Standard Deviation
print("Variance:\n", df.var(numeric_only=True))
print("Standard Deviation:\n", df.std(numeric_only=True))
```

```
Variance:
 Student_ID           77.500000
Age                    2.409195
Study_Hours            4.874713
Attendance           146.322989
Assignment_Score     232.717241
Exam_Score           262.741379
dtype: float64
Standard Deviation:
 Student_ID           8.803408
Age                   1.552158
Study_Hours           2.207875
Attendance           12.096404
Assignment_Score     15.255073
Exam_Score           16.209299
dtype: float64
```

PYTHON CODE:

```
# 4. Quartiles and IQR
Q1 = df['Exam_Score'].quantile(0.25)
Q2 = df['Exam_Score'].quantile(0.50)
Q3 = df['Exam_Score'].quantile(0.75)
IQR =Q3 - Q1
print("Q1, Q2, Q3:", Q1, Q2, Q3)
print("IQR:", IQR)
```

```
Q1, Q2, Q3: 56.5 74.5 82.5
IQR: 26.0
```

PYTHON CODE:

```
# 5. Percentiles
print("Percentiles:\n", np.percentile(df['Exam_Score'], [25, 50, 75, 90]))
```

```
Percentiles:
 [56.5 74.5 82.5 91.1]
```

PYTHON CODE:

```
# 6. Skewness and Kurtosis
print("Skewness:\n", df.skew(numeric_only=True))
print("Kurtosis:\n", df.kurt(numeric_only=True))
```

```
Skewness:
 Student_ID          0.000000
Age                 -0.232553
Study_Hours         -0.177343
Attendance           0.244727
Assignment_Score     0.061260
Exam_Score          -0.179522
dtype: float64
Kurtosis:
 Student_ID          -1.200000
Age                 -0.933356
Study_Hours         -1.108567
Attendance          -1.221753
Assignment_Score    -1.229378
Exam_Score          -1.016788
dtype: float64
```

PYTHON CODE:

```
# 7. Covariance
print("Covariance:\n", df[['Study_Hours', 'Exam_Score']].cov())
```

```
Covariance:
              Study_Hours  Exam_Score
Study_Hours     4.874713    8.982759
Exam_Score      8.982759  262.741379
```

PYTHON CODE:

```
# 8. Pearson Correlation
print("Pearson Correlation:", df['Study_Hours'].corr(df['Exam_Score']))
```

```
Pearson Correlation: 0.25099841676541573
```

PYTHON CODE:

```
# 9. Outliers using IQR
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[(df['Exam_Score'] < lower) | (df['Exam_Score'] > upper)]
print("Outliers:\n", outliers)
```

```
Outliers:
 Empty DataFrame
Columns: [Student_ID, Age, Study_Hours, Attendance, Assignment_Score, Exam_Score]
Index: []
```

PYTHON CODE:

```
# 10. Statistical summary and visualizations
print("Statistical Summary:\n", df.describe())

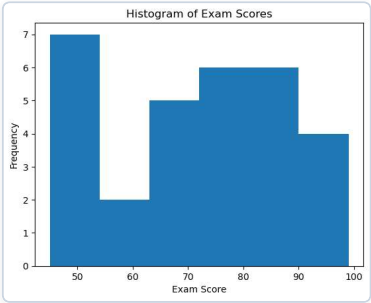
plt.hist(df['Exam_Score'], bins=6)
plt.xlabel('Exam Score')
plt.ylabel('Frequency')
plt.title('Histogram of Exam Scores')
plt.show()

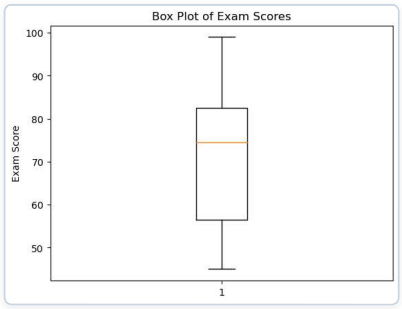
plt.boxplot(df['Exam_Score'])
plt.ylabel('Exam Score')
plt.title('Box Plot of Exam Scores')
plt.show()

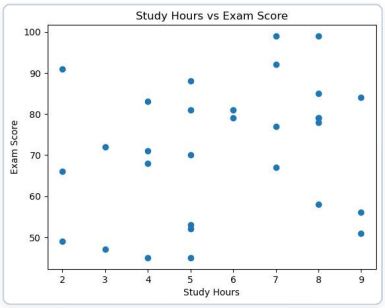
plt.scatter(df['Study_Hours'], df['Exam_Score'])
plt.xlabel('Study Hours')
plt.ylabel('Exam Score')
plt.title('Study Hours vs Exam Score')
plt.show()
```

```
Statistical Summary:
       Student_ID        Age  Study_Hours  Attendance  Assignment_Score  \
count   30.000000  30.000000    30.000000   30.000000         30.000000
mean    15.500000  20.733333     5.766667   78.233333         74.200000
std      8.803408   1.552158     2.207875   12.096404         15.255073
min      1.000000  18.000000     2.000000   61.000000         50.000000
25%      8.250000  20.000000     4.000000   67.250000         61.000000
50%     15.500000  21.000000     5.500000   76.500000         73.500000
75%     22.750000  22.000000     8.000000   87.750000         88.500000
max     30.000000  23.000000     9.000000   99.000000        100.000000

       Exam_Score
count   30.000000
mean    71.500000
std     16.209299
min     45.000000
25%     56.500000
50%     74.500000
75%     82.500000
max     99.000000
```







Question 6: Linear Regression: Study Hours vs Marks Prediction

Problem Statement:

Implementation of Simple Linear Regression using scikit-learn, train/test split, MSE and R2 evaluation, prediction for 7.5 study hours, and regression line visualization.

PYTHON CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 1. Create dataset
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y = np.array([35, 40, 50, 55, 65, 70, 75, 85, 90, 95])
```

PYTHON CODE:

```
# 2. Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

PYTHON CODE:

```
# 3. Create Linear Regression model
model = LinearRegression()
```

PYTHON CODE:

```
# 4. Train the model
model.fit(X_train, y_train)
```

```
LinearRegression()
```

PYTHON CODE:

```
# 5. Predict test data
y_pred = model.predict(X_test)
```

PYTHON CODE:

```
# 6. Display model parameters
print("Slope (Coefficient):", model.coef_[0])
print("Intercept:", model.intercept_)
```

```
Slope (Coefficient): 6.724137931034483
Intercept: 29.26724137931034
```

PYTHON CODE:

```
# 7. Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R2 Score:", r2)
```

```
Mean Squared Error: 3.7102407847800203
R2 Score: 0.994063614744352
```

PYTHON CODE:

```
# 8. Predict marks for 7.5 hours of study
hours = np.array([[7.5]])
predicted_marks = model.predict(hours)

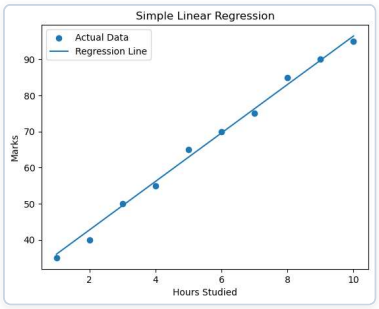
print("Predicted marks for 7.5 hours:", predicted_marks[0])
```

```
Predicted marks for 7.5 hours: 79.69827586206897
```

PYTHON CODE:

```
# 9. Plot data and regression line
plt.scatter(X, y, label="Actual Data")
plt.plot(X, model.predict(X), label="Regression Line")

plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.title("Simple Linear Regression")
plt.legend()
plt.show()
```



Question 7: Multiple Linear Regression: Score Prediction with Multi-Features

Problem Statement:

Predict student exam scores using study hours, attendance and assignment scores, evaluate model performance, and inspect model coefficients and intercept.

PYTHON CODE:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

PYTHON CODE:

```
# Step 1: Create the dataset
data = {
    "Study_Hours": [2, 3, 4, 5, 6, 7, 8, 9, 4, 6],
    "Attendance": [60, 65, 70, 75, 80, 85, 90, 95, 68, 82],
    "Assignment_Score": [55, 60, 65, 70, 75, 80, 85, 90, 62, 78],
    "Exam_Score": [45, 50, 58, 65, 72, 78, 86, 92, 55, 75]
}

df = pd.DataFrame(data)

print("Student Dataset:")
print(df)
```

```
Student Dataset:
   Study_Hours  Attendance  Assignment_Score  Exam_Score
0            2          60                55          45
1            3          65                60          50
2            4          70                65          58
3            5          75                70          65
4            6          80                75          72
5            7          85                80          78
6            8          90                85          86
7            9          95                90          92
8            4          68                62          55
9            6          82                78          75
```

PYTHON CODE:

```
# Step 2: Separate input and output variables
X = df[["Study_Hours", "Attendance", "Assignment_Score"]]
y = df["Exam_Score"]
```

PYTHON CODE:

```
# Step 3: Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

PYTHON CODE:

```
# Step 4: Create and train the model
model = LinearRegression()
model.fit(X_train, y_train)
```

```
LinearRegression()
```

PYTHON CODE:

```
# Step 5: Predict exam scores
y_pred = model.predict(X_test)

print("\nActual Exam Scores:")
print(y_test.tolist())

print("\nPredicted Exam Scores:")
print(y_pred.round(2))
```

```
Actual Exam Scores:
[55, 50, 78]

Predicted Exam Scores:
[55.28 51.56 78.72]
```

PYTHON CODE:

```
# Step 6: Evaluate the model
print("\nMean Squared Error:",
      round(mean_squared_error(y_test, y_pred), 2))

print("R2 Score:", round(r2_score(y_test, y_pred), 2))
```

```
Mean Squared Error: 1.01
R2 Score: 0.99
```

PYTHON CODE:

```
# Step 7: Display coefficients and intercept
print("\nCoefficients:", model.coef_)
print("Intercept:", round(model.intercept_, 2))
```

```
Coefficients: [0.30315789 0.82210526 0.47526316]
Intercept: -31.3
```

PYTHON CODE:

```
# Step 8: Predict for a new student
new_student = pd.DataFrame({
    "Study_Hours": [7],
    "Attendance": [85],
    "Assignment_Score": [82]
})

predicted_score = model.predict(new_student)

print("\nPredicted Exam Score for New Student:",
      round(predicted_score[0], 2))
```

```
Predicted Exam Score for New Student: 79.67
```

Question 8: Logistic Regression: Student Pass/Fail Classification

Problem Statement:

Binary classification model using Logistic Regression to predict Pass or Fail status based on study hours and attendance, with accuracy and confusion matrix evaluation.

PYTHON CODE:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
```

PYTHON CODE:

```
# Step 1: Create the dataset
data = {
    "Study_Hours": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Attendance": [50, 55, 60, 65, 70, 75, 80, 85, 90, 95],
    "Result": [
        "Fail", "Fail", "Fail", "Fail", "Pass",
        "Pass", "Pass", "Pass", "Pass", "Pass"
    ]
}

df = pd.DataFrame(data)
print("Student Dataset:")
print(df)
```

```
Student Dataset:
   Study_Hours  Attendance Result
0            1          50   Fail
1            2          55   Fail
2            3          60   Fail
3            4          65   Fail
4            5          70   Pass
5            6          75   Pass
6            7          80   Pass
7            8          85   Pass
8            9          90   Pass
9           10          95   Pass
```

PYTHON CODE:

```
# Step 2: Encode the target variable
df["Result"] = df["Result"].map({"Fail": 0, "Pass": 1})
```

PYTHON CODE:

```
# Step 3: Separate input and output variables
X = df[["Study_Hours", "Attendance"]]
y = df["Result"]
```

PYTHON CODE:

```
# Step 4: Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
```

PYTHON CODE:

```
# Step 5: Create and train the model
model = LogisticRegression()
model.fit(X_train, y_train)
```

```
LogisticRegression()
```

PYTHON CODE:

```
# Step 6: Predict the results
y_pred = model.predict(X_test)

print("\nActual Results:")
print(y_test.tolist())

print("\nPredicted Results:")
print(y_pred.tolist())
```

```
Actual Results:
[0, 1, 1]

Predicted Results:
[0, 1, 1]
```

PYTHON CODE:

```
# Step 7: Evaluate the model
print("\nAccuracy:",
      round(accuracy_score(y_test, y_pred) * 100, 2), "%")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Accuracy: 100.0 %

Confusion Matrix:
[[1 0]
 [0 2]]
```

PYTHON CODE:

```
# Step 8: Predict for a new student
new_student = pd.DataFrame({
    "Study_Hours": [6],
    "Attendance": [78]
})

prediction = model.predict(new_student)[0]

result = "Pass" if prediction == 1 else "Fail"
print("\nPredicted Result for New Student:", result)
```

```
Predicted Result for New Student: Pass
```

Question 9: Decision Tree: Result Prediction & Tree Graph Visualization

Problem Statement:

Decision Tree Classifier to predict student outcome based on study hours, attendance, and assignment score, with visual tree rendering and test predictions.

PYTHON CODE:

```
# Step 1: Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
```

PYTHON CODE:

```
# Step 2: Create the dataset
data = {
    'Study_Hours': [2, 5, 1, 6, 3, 7, 2, 8, 4, 6],
    'Attendance': [60, 85, 50, 90, 70, 95, 55, 98, 75, 88],
    'Assignment_Score': [40, 80, 30, 90, 50, 95, 35, 98, 65, 85],
    'Result': [
        'Fail', 'Pass', 'Fail', 'Pass', 'Fail',
        'Pass', 'Fail', 'Pass', 'Pass', 'Pass'
    ]
}

df = pd.DataFrame(data)
```

PYTHON CODE:

```
# Step 3: Display the dataset
print("Student Dataset:")
print(df)
```

```
Student Dataset:
   Study_Hours  Attendance  Assignment_Score Result
0            2          60                40   Fail
1            5          85                80   Pass
2            1          50                30   Fail
3            6          90                90   Pass
4            3          70                50   Fail
5            7          95                95   Pass
6            2          55                35   Fail
7            8          98                98   Pass
8            4          75                65   Pass
9            6          88                85   Pass
```

PYTHON CODE:

```
# Step 4: Separate features (X) and target (y)
X = df[['Study_Hours', 'Attendance', 'Assignment_Score']]
y = df['Result']
```

PYTHON CODE:

```
# Step 5: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)
```

PYTHON CODE:

```
# Step 6: Train the Decision Tree Classifier
model = DecisionTreeClassifier(
    criterion='entropy',
    random_state=42
)

model.fit(X_train, y_train)
```

```
DecisionTreeClassifier(criterion='entropy', random_state=42)
```

PYTHON CODE:

```
# Step 7: Predict student results
y_pred = model.predict(X_test)

print("\nActual Results:", list(y_test))
print("Predicted Results:", list(y_pred))
```

```
Actual Results: ['Fail', 'Pass', 'Pass']
Predicted Results: ['Fail', 'Pass', 'Pass']
```

PYTHON CODE:

```
# Step 8: Evaluate the model
accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:", accuracy)
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
```

```
Model Accuracy: 1.0

Classification Report:
              precision    recall  f1-score   support

        Fail       1.00      1.00      1.00         1
        Pass       1.00      1.00      1.00         2

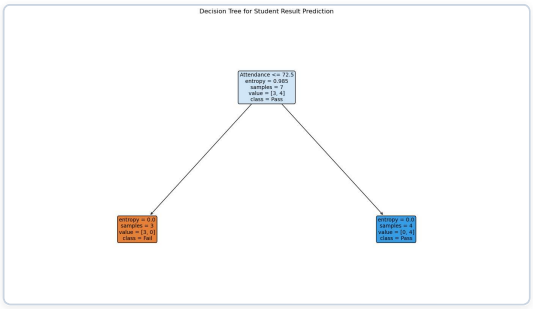
    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3
```

PYTHON CODE:

```
# Step 9: Visualize the decision tree
plt.figure(figsize=(14, 8))

plot_tree(
    model,
    feature_names=list(X.columns),
    class_names=[str(cls) for cls in model.classes_],
    filled=True,
    rounded=True,
    fontsize=10
)

plt.title("Decision Tree for Student Result Prediction")
plt.tight_layout()
plt.show()
```



PYTHON CODE:

```
# Step 10: Predict the result for a new student
new_student = pd.DataFrame({
    'Study_Hours': [5],
    'Attendance': [80],
    'Assignment_Score': [75]
})

prediction = model.predict(new_student)

print("\nNew Student Details:")
print(new_student)

print("Predicted Student Result:", prediction[0])
```

```
New Student Details:
   Study_Hours  Attendance  Assignment_Score
0            5          80                75
Predicted Student Result: Pass
```